

Primitives for Non-Local Concurrency

Document #: P4215R0 [[Latest](#)] [[Status](#)]
Date: 2026-05-12
Project: Programming Language C++
Audience: SG1
Reply-to: Lucian Radu Teodorescu (Garmin)
<lucteo@lucteo.ro>

Contents

- [1 Introduction](#)
 - [1.1 The essence of concurrency](#)
 - [1.2 Local and non-local concurrency](#)
 - [1.3 Correctness: Safety + Progress](#)
 - [1.4 A principled way to design concurrent abstractions](#)
- [2 Old synchronization primitives](#)
 - [2.1 Mutex](#)
 - [2.2 Shared mutex](#)
 - [2.3 Timed and recursive mutexes](#)
 - [2.4 `try\_lock` facilities](#)
 - [2.5 Semaphores](#)
 - [2.6 Condition variables](#)
 - [2.7 Latches](#)
 - [2.8 Barriers](#)
- [3 Primitives for non-local concurrency](#)
 - [3.1 `serial\_gate`](#)
 - [3.1.1 Constraints](#)
 - [3.1.2 API](#)
 - [3.1.3 Alternative API](#)
 - [3.1.4 Examples](#)
 - [3.1.5 Safety and progress](#)

- 3.2 read_write_gate
 - 3.2.1 Constraints
 - 3.2.2 API
 - 3.2.3 Examples
 - 3.2.4 Safety and progress
- 3.3 capacity_gate
 - 3.3.1 Constraints
 - 3.3.2 API
 - 3.3.3 Examples
 - 3.3.4 Safety and progress
- 3.4 Speculative execution
- 3.5 readiness_gate
 - 3.5.1 Constraints
 - 3.5.2 API
 - 3.5.3 Alternative API: predicate-owned readiness
 - 3.5.4 Examples
 - 3.5.5 Safety and progress
- 3.6 completion_gate
 - 3.6.1 Constraints
 - 3.6.2 API
 - 3.6.3 Examples
 - 3.6.4 Safety and progress
- 3.7 phase_gate
 - 3.7.1 Constraints
 - 3.7.2 API
 - 3.7.3 Examples
 - 3.7.4 Safety and progress
- 3.8 Destruction
- 3.9 Beyond classic-inspired gates
- 4 Relation to other facilities
 - 4.1 P3955R0: asynchronous scopes
 - 4.2 counting_scope
 - 4.3 Other related facilities

§ Abstract

C++26 senders provide a vocabulary for composing asynchronous work, but they do not by themselves provide sender-native equivalents of synchronization primitives that impose non-local concurrency constraints. Programs still need to serialize unrelated operations, bound concurrency across independently submitted work, wait for readiness, and coordinate completion or phase boundaries.

This paper explores a family of sender-based primitives, called *gates*, that control admission of work according to such non-local constraints. The design is motivated by the ordering relations expressed by existing synchronization primitives, but aims to preserve sender composition, safety invariants, progress, non-blocking waiting, and compatibility with structured lifetime management.

The paper is exploratory. It asks SG1 whether this model is a useful direction for C++ concurrency, and where further design effort should be focused.

§ 1 Introduction

The sender model gives C++ a way to describe asynchronous work as values. Sender algorithms allow programs to compose work into execution graphs, and completion signals make the flow of values, errors, and cancellation explicit. This is a significant improvement over ad-hoc asynchronous APIs.

Most sender composition is local: the work items being related are present in the same expression, or are connected by an execution graph that can be inspected from one place. For example, a program can express that two operations run concurrently with `when_all`, that one operation starts after another with `let_value`, or that work transfers to a scheduler with `starts_on` or `continues_on`.

Some concurrency constraints are not local in this sense. Consider an application in which several subsystems can write to the same file. The subsystem that initiates one write may not know about the subsystem that initiates another write. Nevertheless, the program must impose a non-local constraint: two writes to that file must not execute concurrently. Similar constraints occur when independent request paths share a database connection pool, when many operations depend on a one-time initialization step, or when a set of workers must rendezvous at a phase boundary.

In synchronous code, we routinely express these constraints with synchronization objects. A mutex is not local to one call graph; it is a shared object through which otherwise independent pieces of code participate in the same serialization constraint. A semaphore

bounds concurrent access to a resource. A condition variable coordinates progress based on shared state. A latch or barrier establishes ordering between groups of work.

The question for sender-based C++ is not whether these constraints still exist. They do. The question is how to express them without falling back to blocking threads, manual signaling protocols, or unstructured lifetime management.

This paper approaches the question from the constraints themselves. We first look at classic synchronization primitives as ways to impose ordering or concurrency relations between work items. We then explore sender-native objects, called *gates*, that express similar non-local constraints while composing with sender algorithms, cancellation, completion signaling, and asynchronous scopes.

§ 1.1 The essence of concurrency

Serial execution imposes a total order on the execution of work items. If a and b are two work items, then either $a < b$ or $b < a$.

On the other hand, concurrent execution imposes a strict partial ordering over the execution of work items. Thus, we have 3 possibilities for executing two work items a and b :

- $a < b$;
- $b < a$;
- neither $a < b$ nor $b < a$ (concurrent execution).

This is a run-time view of the execution. At program specification, it is useful to consider another relation:

- either $a < b$ or $b < a$ (mutual exclusion)

We denote this last relation with $a < > b$. For convenience, we also denote with $a \parallel b$ the absence of any imposed ordering relation between a and b – this does not guarantee that they would run concurrently; they may or may not execute concurrently.

In this document we always use the “ $<$ ” symbol to denote ordering between work items. We do not use it as the less operator from C++.

Typically any constraint of the form $a < b$ means *a happens-before b*. However, the model may be relaxed to consider other types of relations.

For more details please see [\[Teodorescu26\]](#).

§ 1.2 Local and non-local concurrency

We call a concurrency constraint *local* when the work items being constrained are composed together directly. For example:

```

auto s
= std::execution::when_all(read_header(), read_body())
| std::execution::let_value([](auto header, auto body) {
    return parse_message(header, body);
});

```

The relationship between `read_header` , `read_body` , and `parse_message` is local to the sender expression. The expression itself describes which work can overlap and which work depends on earlier completion. In this fragment, we have: `read_header || read_body` , and `read_header < parse_message` , and `read_body < parse_message` .

We call a concurrency constraint *non-local* when the constrained work items are not necessarily composed together at the same point in the program. The constraint is instead mediated by a shared object, policy, or resource. For example:

```

void subsystem_a() {
    write_log_record(...);
}

void subsystem_b() {
    write_log_record(...);
}

```

The two calls may be initiated by unrelated parts of the program, but they may still need to participate in the same serialization constraint if they write to the same file. The constraint is non-local because it is not visible in either call site alone.

While we can easily encode local constraints with sender algorithms, encoding non-local constraints is not trivial. Reasoning about non-local constraints is also harder, and we do not, in general, have guarantees that progress composes.

At its core, non-local concurrency control is the *repeated application of a partition function over the set of work items*. In other words, we need to decide what can start execution at any given time, and what needs to be postponed.

The important observation is that most programs do not need a single global concurrency policy. Concurrency constraints are usually sparse. A log file, a database connection pool, a readiness condition, or a phase boundary constrains only the work items that use that object. Work outside that object is unrelated.

Thus, we can model each such object as a small admission controller for one non-local constraint. The controller partitions associated work into work that may run now and work that must wait. In synchronous code, this role is played by mutexes, semaphores, condition variables, latches, and barriers. In sender-based asynchronous code, we want corresponding facilities that impose the same kinds of constraints without blocking execution resources and without exposing manual release or notification protocols when sender structure can express them.

This paper calls these sender-native admission controllers *gates*. A gate is shared by otherwise independent sender expressions, and it imposes a named non-local concurrency con-

straint on the work associated with it.

Some of these abstractions have a long history in practice. Task queues, strands, serializers, asynchronous semaphores, rate limiters, and similar facilities are widely used in asynchronous systems. This paper is not primarily trying to claim novelty for each individual primitive. Instead, it tries to identify the underlying concurrency relations that such facilities impose, and to describe a common sender-native model that can extend from familiar cases to less-established non-local constraints.

A more detailed treatment of this model can be found in [\[Teodorescu26\]](#).

§ 1.3 Correctness: Safety + Progress

This paper uses the terminology from [\[P4214R0\]](#): correctness is decomposed into safety and liveness, and for the class of concurrency facilities discussed here, liveness is treated through progress. A useful concurrency primitive should therefore preserve both:

- **safety**: the primitive must maintain the invariants associated with the work items it executes and the constraint it represents;
- **progress**: the primitive must not introduce hidden dependencies that can prevent otherwise-progressing work from completing.

Thus, throughout this paper, the safety and progress discussion for each gate asks two questions:

- What invariant must the gate maintain over the work associated with it?
- Under what assumptions should waiting operations eventually be admitted or otherwise resolved?

§ 1.4 A principled way to design concurrent abstractions

The rest of this paper follows a simple method.

First, we identify the relation imposed by existing synchronization primitives. For example, a mutex imposes an ordering constraint between critical sections; a semaphore imposes a cardinality constraint on concurrently executing work; a barrier imposes ordering between phases.

Second, we ask how the same kind of relation could be expressed for sender-based asynchronous work. The resulting abstraction should not block an execution resource while waiting. It should compose with sender algorithms and completion signals. It should avoid manual release or notification protocols when release can instead be structurally tied to sender completion.

Third, we evaluate whether the abstraction preserves safety and progress compositionally. If the submitted work is safe and makes progress when executed under the intended con-

straint, the primitive should not introduce hidden races, lost notifications, or progress dependencies unrelated to the submitted work.

The following sections apply this method to the gate vocabulary introduced above.

§ 2 Old synchronization primitives

Here we analyze synchronization primitives in the C++ standard from a pre-senders perspective.

§ 2.1 Mutex

Let us look at the following example using an `std::mutex` :

```
std::mutex m;

void usage() {
    head();
    {
        std::lock_guard<std::mutex> guard{m};
        w();
    }
    tail();
}
```

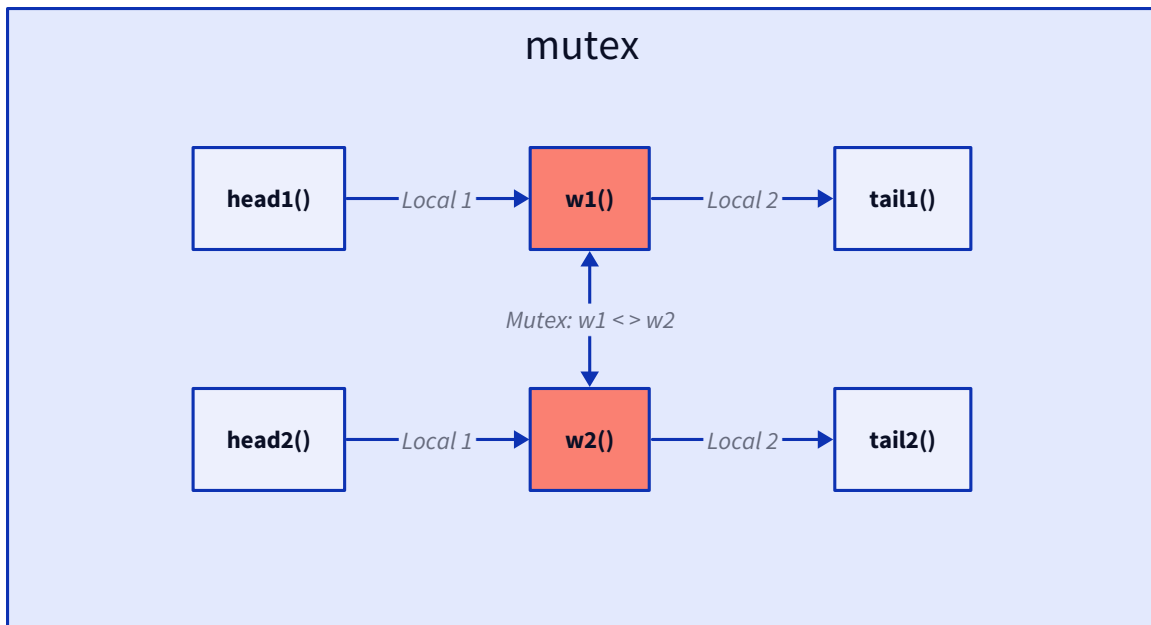
We execute work `w` in a region protected by mutex `m` , with `head` being executed before `w` and `tail` being executed after it.

Thus, we have the following ordering constraints:

- `head < w` (*Local 1*)
- `w < tail` (*Local 2*)
- `head < tail` (*Local 3*)

In addition to these, the mutex imposes the extra set of ordering constraints:

- if `w1` and `w2` are protected by `m` , then `w1 < > w2` (*Mutex*)



The constraints imposed by a mutex.

§ 2.2 Shared mutex

Similar to the previous example, we can look at an example that uses shared mutexes:

```
std::shared_mutex m;

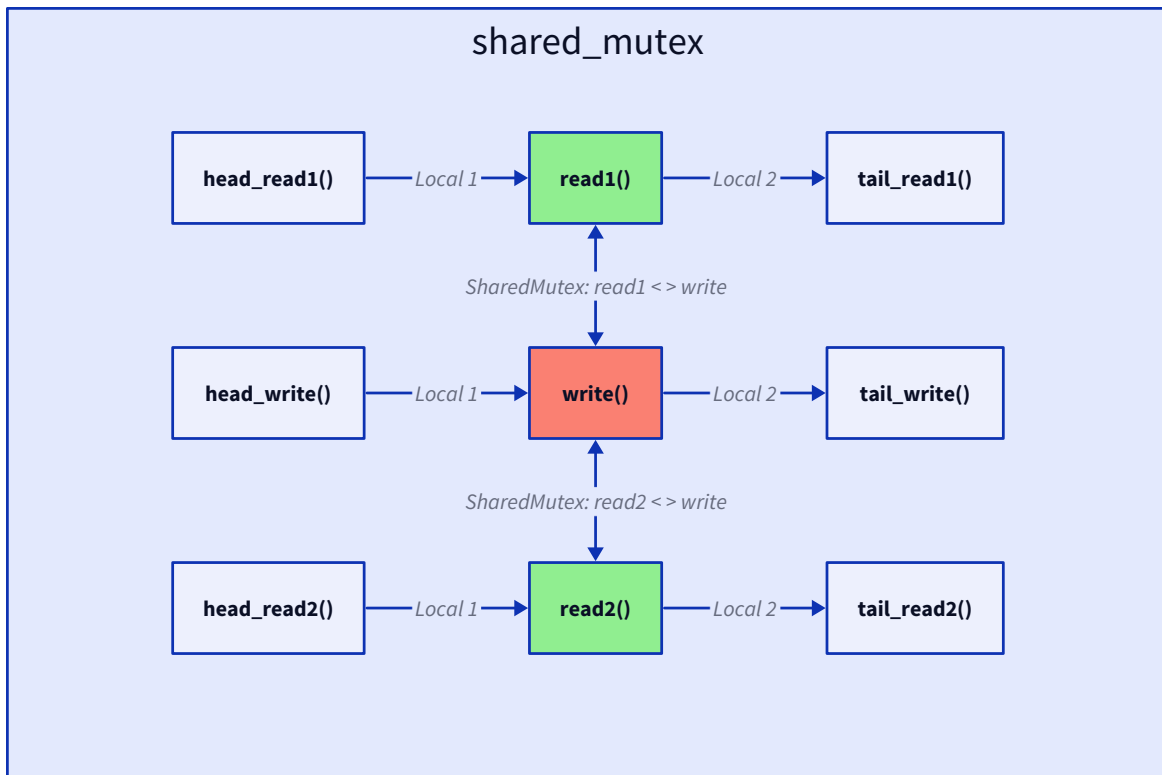
void usage_read() {
    head();
    {
        std::shared_lock guard{m};
        w();
    }
    tail();
}

void usage_write() {
    head();
    {
        std::unique_lock guard{m};
        w();
    }
    tail();
}
```

A shared mutex can be used in two modes: for reading data (shared mode), or for writing data (unique mode). The *Local* ordering constraints discussed for mutexes also apply for shared mutexes, regardless of the mode in which they are used. The core ordering constraint from mutex morphs into:

- if $w1$ and $w2$ are protected by m , either of which being used in write mode, then $w1 < w2$ (*SharedMutex*)

As expected, multiple read operations may execute concurrently, but a read cannot happen concurrently with a write, and neither can two writes.



The constraints imposed by a `shared_mutex`.

§ 2.3 Timed and recursive mutexes

Timed and recursive mutexes do not introduce new ordering constraints compared to the basic mutex. Thus, they are not interesting for our analysis.

§ 2.4 `try_lock` facilities

The `try_lock` family of functions allows the user to execute a work item only if other work items on the mutex are not executing at that time. Compared to our previous example, `w` might not execute. When `w` is executed, then all (*Local*) ordering constraints need to be followed; otherwise, (*Local 1*) and (*Local 2*) make no sense.

§ 2.5 Semaphores

A `counting_semaphore` behaves like a mutex, but allows more than one concurrent access to the same resource. Also, unlike a mutex, the semaphore allows unstructured access to the resource; the resource does not have ownership like in the mutex case. Semaphores can be used in signaling mechanisms in which mutexes are harder to use (e.g., cross-thread handoff). Conceptually, a mutex protects a resource while a semaphore coordinates progress.

A `binary_semaphore` is a semaphore that cannot have more than one concurrent access to the resource. From that perspective it is similar to a mutex, but it allows the unstructured access patterns of a `counting_semaphore`.

The useful constraint provided by a semaphore is the concurrency bound. The problematic part, for our purposes, is not bounded concurrency itself, but the unstructured manual acquire/release protocol: a release can be forgotten, performed on only one completion path, or disconnected from the work whose completion should free capacity.

Let us take the following example as being representative for a semaphore:

```
std::counting_semaphore s(10);

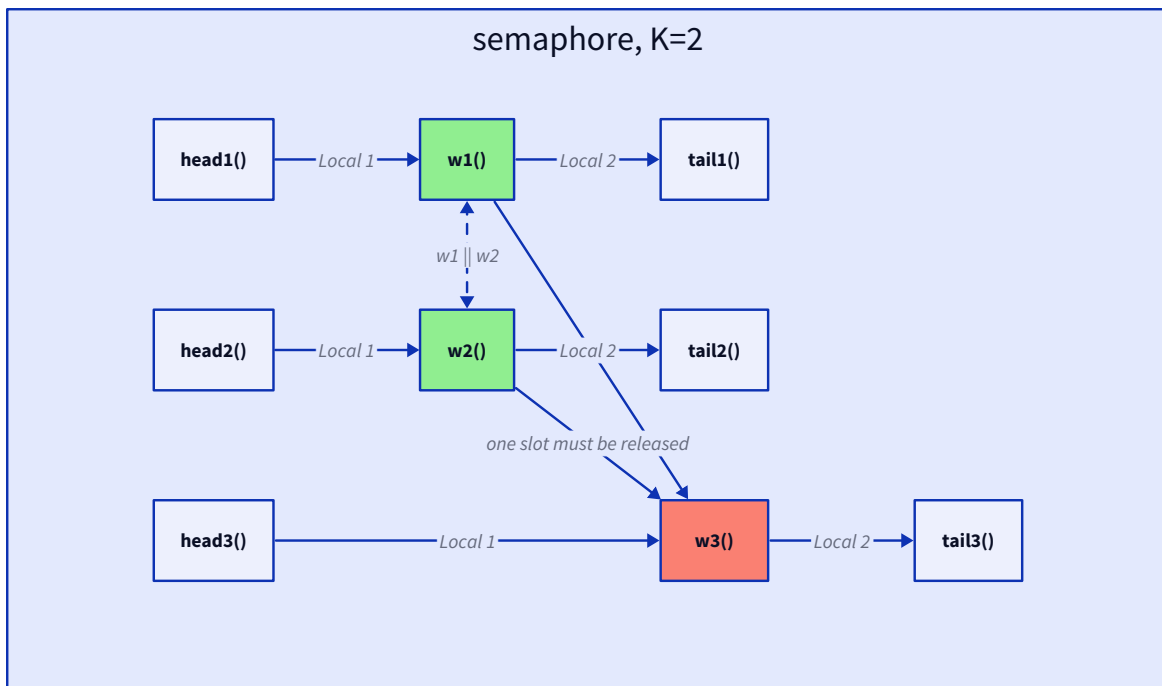
void usage(int index) {
    head();
    s.acquire();
    w(index);
    s.release();
    tail();
}
```

To discuss ordering constraints, let $W = \{ w_1, w_2, \dots, w_N \}$ be the set of work items protected by semaphore s with count K .

Like in the previous cases, the *Local* ordering constraints also apply here. On top of those, we have something specific to semaphores:

- for any $W_0 \subset W$ so that $w_i \parallel w_j \forall i, j \in W_0$, the cardinality of W_0 is not greater than K (*Semaphore*)

In other words, we cannot have more than K work items execute concurrently. If we have more work items that are ready to be executed, their execution needs to be delayed until one of the currently K work items completes. This is a concurrency cardinality constraint.



The constraints imposed by a counting_semaphore.

§ 2.6 Condition variables

A condition variable suspends the execution of a thread of execution until a condition is met. A condition variable is always paired with a mutex that is used to protect access to the data underlying the condition that is blocking execution.

The following is an illustrative example of a condition variable:

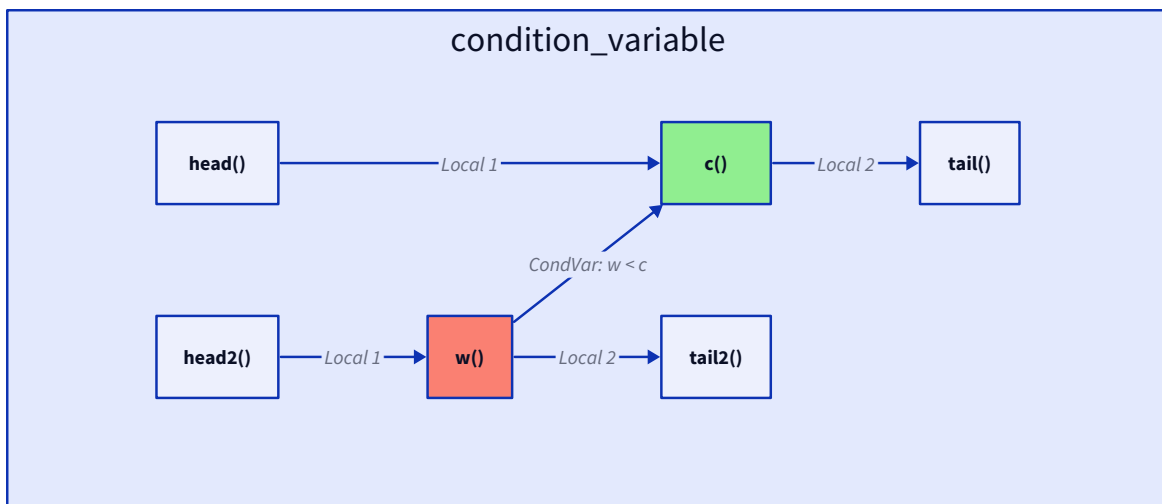
```
std::mutex m;
std::condition_variable cv;

void condition_check(int index) {
    head();
    std::unique_lock lock(m);
    cv.wait(lock, [] { return c(); });
    tail();
}

void update_condition(int index) {
    head2();
    {
        std::unique_lock lock(m);
        w();
    }
    cv.notify_one();
    tail2();
}
```

Let us try to extract the ordering constraints for this example. First, both parts (checking the condition and updating the condition) follow the *Local* ordering constraints. We also have the *Mutex* ordering constraint apply for checking and updating the condition. In addition to those, we also have extra ordering constraints for condition variables:

- assuming the condition was initially false, the successful evaluation of `c()` that allows `cv.wait` to complete observes an update performed by some `w`, so that $w < c$ (*CondVar*)



The constraints imposed by a condition_variable.

§ 2.7 Latches

A latch allows one or more threads of execution to block until a fixed number of arrivals has occurred. Once the expected number of arrivals has been reached, the latch stays open and cannot be reused.

Let us take the following example as being representative for a latch:

```
std::latch l(K);

void worker(int index) {
    head(index);
    w(index);
    l.count_down();
    tail(index);
}

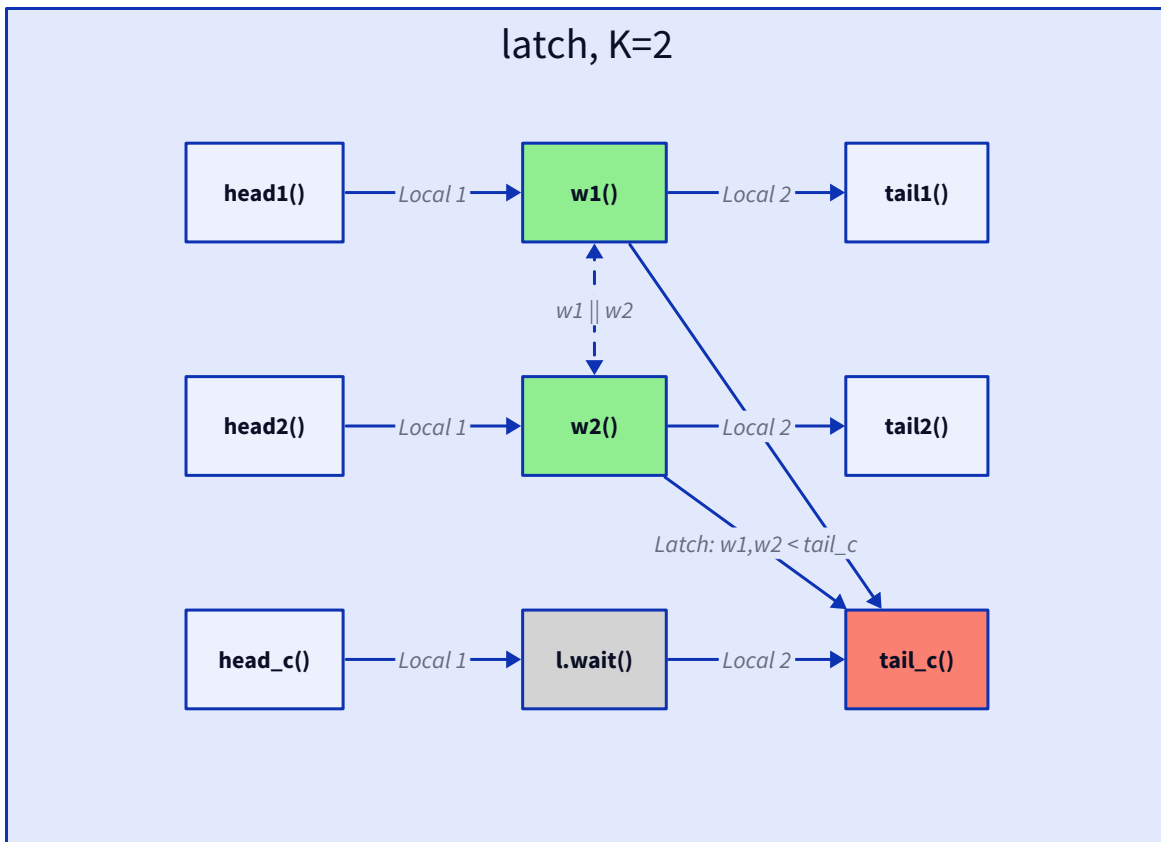
void coordinator() {
    head_c();
    l.wait();
    tail_c();
}
```

To discuss ordering constraints, let $W = \{ w_1, w_2, \dots, w_K \}$ be the set of work items whose arrivals cause latch l to reach zero.

Like in the previous cases, the *Local* ordering constraints apply for each worker and for the coordinator. On top of those, we have something specific to latches:

- if $w \in W$, then $w < \text{tail_c}$ (*Latch*)

In other words, all work items that contribute to opening the latch need to complete before the work that follows `l.wait()` can execute. The latch does not impose ordering constraints between the work items in W themselves; they can execute concurrently.



The constraints imposed by a latch.

§ 2.8 Barriers

A barrier allows a fixed number of threads of execution to repeatedly rendezvous at the end of a phase. Once all participants arrive at the barrier, the phase is completed and the participants can continue with the next phase.

Let us take the following example as being representative for a barrier:

```

std::barrier b(K);

void participant(int index) {
    head1(index);
    w1(index);
    b.arrive_and_wait();
    tail1(index);

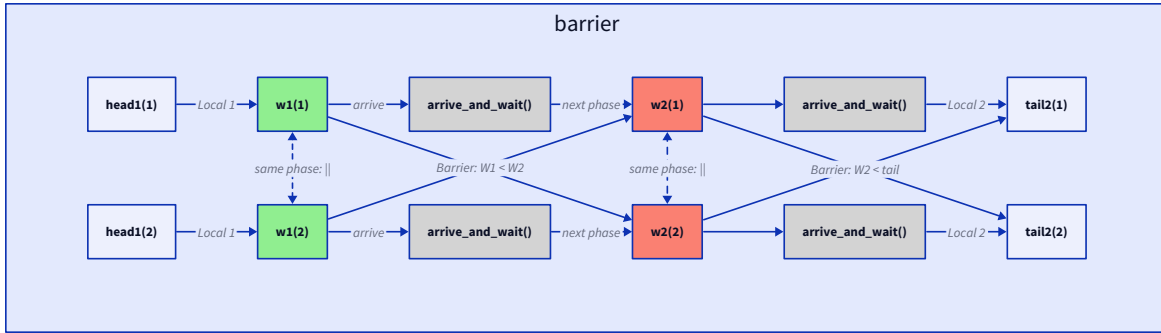
    head2(index);
    w2(index);
    b.arrive_and_wait();
    tail2(index);
}
  
```

To discuss ordering constraints, let $W1 = \{ w1_1, w1_2, \dots, w1_K \}$ be the set of work items executed before the first barrier phase is completed, and let $W2 = \{ w2_1, w2_2, \dots, w2_K \}$ be the set of work items executed after the first barrier phase is completed.

Like in the previous cases, the *Local* ordering constraints apply for each participant. On top of those, we have something specific to barriers:

- if $w1 \in W1$ and $w2 \in W2$, then $w1 < w2$ (*Barrier*)

In other words, a barrier imposes an ordering constraint between phases: all work items before a barrier phase need to complete before any work item after that barrier phase can execute. The barrier does not impose ordering constraints between work items in the same phase; they can execute concurrently.



The constraints imposed by a barrier.

§ 3 Primitives for non-local concurrency

The synchronization primitives above allow programs to impose ordering and concurrency constraints between work items that are not locally related in the call graph. Mutexes serialize critical sections, semaphores bound concurrency, condition variables coordinate progress, and latches and barriers establish cross-thread phase boundaries.

The sender model gives C++ a vocabulary for structured asynchronous work, but it does not by itself provide replacements for these non-local constraints. This paper therefore considers primitives that can express such constraints while remaining compatible with structured concurrency.

We use the following goals to evaluate candidate primitives:

- **Preserve safety compositionally.** A primitive should not require users to manually maintain hidden ordering or signaling protocols whose safety depends on all participating operations being written together. If each operation is locally safe, combining operations through the primitive should not introduce data races, lost wakeups, or invariant violations.
- **Provide composable progress.** A primitive should not make forward progress depend on an unrelated operation eventually performing a manual signal, release, or notification unless that dependency is explicit in the sender graph. In particular, candidate designs should avoid progress dependencies that are invisible to cancellation, shutdown, and structured lifetime management.
- **Integrate with senders.** The primitive should expose sender-based operations, so that use of the primitive composes with existing sender algorithms, cancellation,

completion signaling, and scheduler selection.

- **Avoid blocking execution resources.** Waiting for a non-local concurrency constraint should suspend the operation, not block an operating-system thread or other scarce execution resource. Resources should be consumed only when useful work can be performed.
- **Avoid introducing latency delays.** If possible, using our primitives should not tend to increase latency for operations if there is outstanding work that is not related to the current scope.
- **Make the constraint explicit.** The primitive should make the imposed constraint visible in the program structure: serialization, bounded concurrency, phase ordering, or another named relation between work items.

Proposed primitive	Alternative names	Corresponding legacy primitive
serial_gate	task_queue , serializer , strand , async_mutex	mutex
read_write_gate	rw_task_queue , rw_serializer , async_shared_mutex	shared_mutex
capacity_gate	n_serializer , bottleneck , work_limiter , concurrency_limiter , throttler , async_semaphore	counting_semaphore
readiness_gate	condition_signal , condition , state_signal , state_condition , async_condition_variable	condition_variable
completion_gate	countdown_signal , arrival_gate , join_counter , async_latch	latch
phase_gate	phase_signal , phase_join , rendezvous , phase_boundary , async_barrier	barrier

We prefer using the word *gate* in these names, as it suggests admission into a region of work, and it avoids implying blocking threads or legacy synchronization mechanics. It also resonates with the idea of adding (concurrency) constraints.

§ 3.1 serial_gate

Similar to a mutex but for the structured concurrency world, a `serial_gate` ensures that only one work item is executed at a given time.

§ 3.1.1 Constraints

The `serial_gate` abstraction needs to maintain the (*Mutex*) constraint. In addition to that, we want to maintain as much as possible from the (*Local*) constraints. However, satisfying all of them is not ideal.

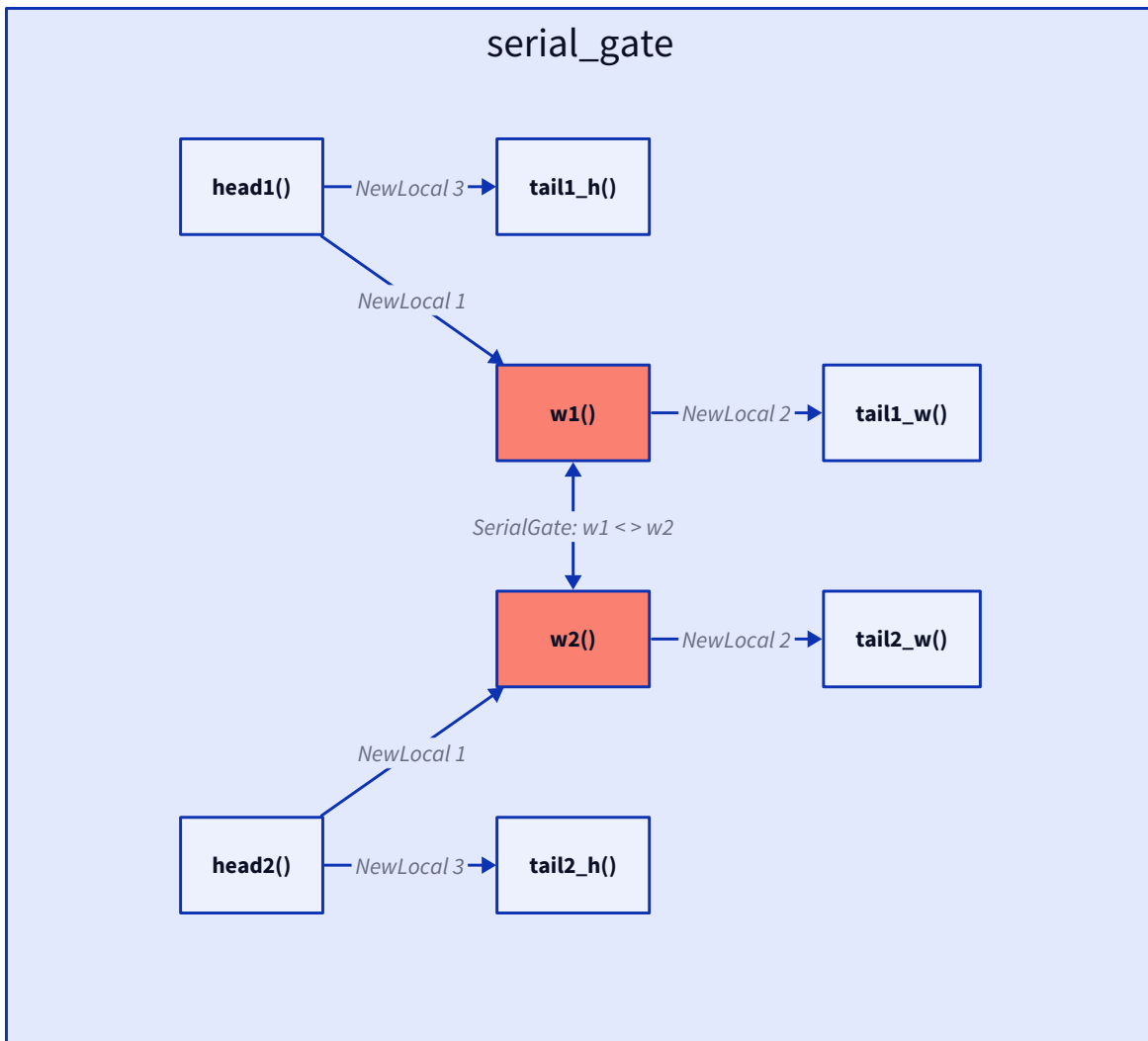
If we were to satisfy all of the local constraints, `tail` would be forced to wait until `w` is complete. Because there are no other instructions between `head`, the protected execution of `w`, and `tail`, this wait implies either blocking the thread or executing other work. Blocking is not acceptable for an asynchronous primitive. Executing unrelated work is also not ideal, as it can increase the latency of `tail`: if the current thread starts executing unrelated work, it may need to wait until that work completes before it can execute `tail`.

The best option is to split `tail` into two work items: what needs to be executed after `head` (*Local 3*), and what needs to be executed after `w` (*Local 2*). We call these work items `tail_h` and `tail_w` respectively.

With this split, we have a new set of local constraints:

- `head < w` (*NewLocal 1*)
- `w < tail_w` (*NewLocal 2*)
- `head < tail_h` (*NewLocal 3*)

The following image shows how the constraints look like for `serial_gate`:



The constraints imposed by a serial_gate.

§ 3.1.2 API

Based on the ideas from [P3955R0], an API might look like:

```

struct serial_gate {
    std::execution::enter_scope_sender acquire();
};

```

The users will be able to use it like:

```

serial_gate g;
std::execution::counting_scope scope;

void usage() {
    head();

    auto s = std::execution::within(g.acquire(), w_sender())
        | std::execution::then([] { tail_w(); });

    std::execution::spawn(std::move(s), scope.get_token());

    tail_h();
}

```

This example assumes that the spawned sender does not complete with `set_error` ; otherwise the error must be handled before calling `spawn` .

§ 3.1.3 Alternative API

Inspired from the API of `counting_scope` , we can have the following API:

```
struct serial_gate {  
    struct token; // models std::execution::scope_token  
  
    token get_token() noexcept;  
};
```

This can be used the following way:

```
serial_gate g;  
std::execution::counting_scope scope;  
  
void usage() {  
    head();  
  
    auto s = std::execution::associate(w_sender(), g.get_token())  
        | std::execution::then([] { tail_w(); });  
    std::execution::spawn(std::move(s), scope.get_token());  
  
    tail_h();  
}  
void usage2() { // without tail_w  
    head();  
  
    std::execution::spawn(w_sender(), g.get_token());  
  
    tail_h();  
}
```

For the purposes of this paper, we are not interested in exploring the API design alternative. We simply assume that the previous design is the preferred one.

§ 3.1.4 Examples

The following examples use the `acquire()` API from the previous section. For brevity, they assume that spawned senders either do not complete with `set_error` or handle errors before being passed to `spawn` , and that the enclosing program eventually joins the `counting_scope` .

§ 3.1.4.1 Protecting non-thread-safe state

A `serial_gate` can be used to protect state that is accessed by asynchronous operations. Unlike a `mutex` , entering the protected region suspends the operation until it can make progress; it does not block the current thread of execution.

```

struct cache {
    void insert(record);
};

serial_gate cache_gate;
cache c;
std::execution::counting_scope scope;

void on_record(record r) {
    auto update
        = std::execution::just(std::move(r))
        | std::execution::then([&](record r) {
            c.insert(std::move(r));
        });

    auto protected_update =
        std::execution::within(cache_gate.acquire(), std::move(update));

    std::execution::spawn(std::move(protected_update), scope.get_token());
}

```

In this example all calls to `cache::insert` are serialized by `cache_gate`, even if multiple records are received concurrently.

§ 3.1.4.2 Serializing protocol operations

Some asynchronous resources allow many callers, but require only one operation to be active at a time. For example, a connection might require that requests are sent and responses are consumed serially, even though callers can enqueue work concurrently.

```

serial_gate connection_gate;
connection conn;
std::execution::counting_scope scope;

void submit_request(request req) {
    prepare(req); // does not need exclusive access to conn

    auto transaction
        = send_request(conn, std::move(req))
        | std::execution::let_value([&] {
            return read_response(conn);
        })
        | std::execution::then([](response resp) {
            process_response(std::move(resp));
        });

    auto protected_transaction =
        std::execution::within(connection_gate.acquire(), std::move(transaction));

    std::execution::spawn(std::move(protected_transaction), scope.get_token());
}

```

```

    continue_after_submit(); // does not wait for the transaction to finish
}

```

The `serial_gate` constrains only the transaction that uses `conn`. Work before submitting the transaction and work after submission are not forced to wait for the protected operation to complete.

§ 3.1.4.3 Fire-and-forget serialized work

For work that does not need a continuation after the protected region, the protected sender can be spawned directly into an enclosing scope:

```

serial_gate gate;
std::execution::counting_scope scope;

void usage() {
    head();

    auto protected_work =
        std::execution::within(gate.acquire(), w_sender());

    std::execution::spawn(std::move(protected_work), scope.get_token());

    tail_h();
}

```

This form is appropriate when the only required continuation after `w_sender()` is the release of the gate itself.

§ 3.1.5 Safety and progress

A `serial_gate` can be implemented so that it preserves both safety and progress compositionally, assuming that the code submitted to the gate also preserves safety and progress.

For safety, the implementation must maintain the following invariant:

- at most one operation admitted by the gate is executing its protected work at any given time.

This is the sender-based equivalent of the (*Mutex*) constraint described earlier. If each protected work item preserves the invariants of the state it accesses when executed in isolation, then serializing those work items is sufficient to prevent concurrent interference between them. The gate does not need to know which invariants are being protected; it only needs to ensure that no two protected work items can observe or mutate that state concurrently through the gate.

The use of `within(g.acquire(), snd)` is important for this argument. The sender returned by `acquire()` admits the operation to the gate, and the corresponding exit sender releases the gate after `snd` completes. Thus, the release of the gate is tied to the completion of the protected sender, including non-value completion paths. Users are not required to

manually signal the gate after the work is complete, so there is no separate release protocol that can be forgotten or executed in the wrong order.

For progress, a `serial_gate` should avoid blocking an execution resource while an operation is waiting to enter the gate. If the gate is already occupied, the waiting operation is suspended and resumed only when it can make progress. When the current protected operation completes, the implementation selects another waiting operation, if any, and admits it. Under the usual assumptions that admitted work eventually completes and that the scheduler used to resume waiting operations provides progress, the gate itself does not introduce a deadlock or lost wakeup.

In particular, a submitted operation that is waiting to enter the gate should eventually be admitted, provided that the operation is not cancelled or abandoned, the gate remains alive, previously admitted work eventually completes, and the execution agents needed by the implementation continue to make progress. This eventual-admission property is part of what makes the gate useful as a composable concurrency primitive: submitting work to the gate should not create an indefinite wait that is independent of the submitted work and the scheduler.

This is stronger than an asynchronous interface to a semaphore-like primitive with separate acquire and release operations. A user of `serial_gate` is not given an operation that manually releases the gate, so the user cannot forget to release it, release it on only one completion path, or make progress depend on an unrelated notification. Release is structurally connected to completion of the admitted sender.

This progress guarantee is conditional. A `serial_gate` cannot make an operation complete if the protected work never completes, or if the enclosing lifetime mechanism cancels or abandons the operation. Users can also still create progress failures by using the gate in a context that blocks an execution resource needed by the operation, for example by calling `sync_wait` on a scheduler thread that must also run the operation needed to release the gate. It also cannot guarantee fairness unless the specification requires a particular admission policy. These are not failures of the abstraction; they are the same limits that apply to sender-based asynchronous composition in general. The important property is that the gate itself does not expose a manual signaling protocol that lets users break progress independently of the submitted work.

Fairness matters here only for waiter-level progress, not for system-level progress. Even if admitted work keeps completing, a particular waiting operation may still be bypassed indefinitely in an open system where new operations continue to arrive. Therefore, whether a particular waiting operation is eventually admitted depends on the gate's admission policy and any anti-starvation guarantees that policy provides.

We do not plan to require a strict fairness policy, so implementations can optimize admission. Under such a relaxed policy, starvation may be possible; in those cases, per-waiter eventual-admission is not guaranteed. This reflects the scope of the progress contract: the gate guarantees progress only under its stated assumptions; if usage permits starvation un-

der a relaxed admission policy, those assumptions are not met, so per-waiter eventual admission is outside the contract.

Therefore, if the calling code maintains the lifetime of the gate and of the associated operations, and if the protected work items themselves maintain safety and make progress, a `serial_gate` can maintain the non-local serialization constraint without breaking safety or progress.

The destruction rules for all gate objects are discussed in [Destruction](#). They are compatible with the safety and progress goals.

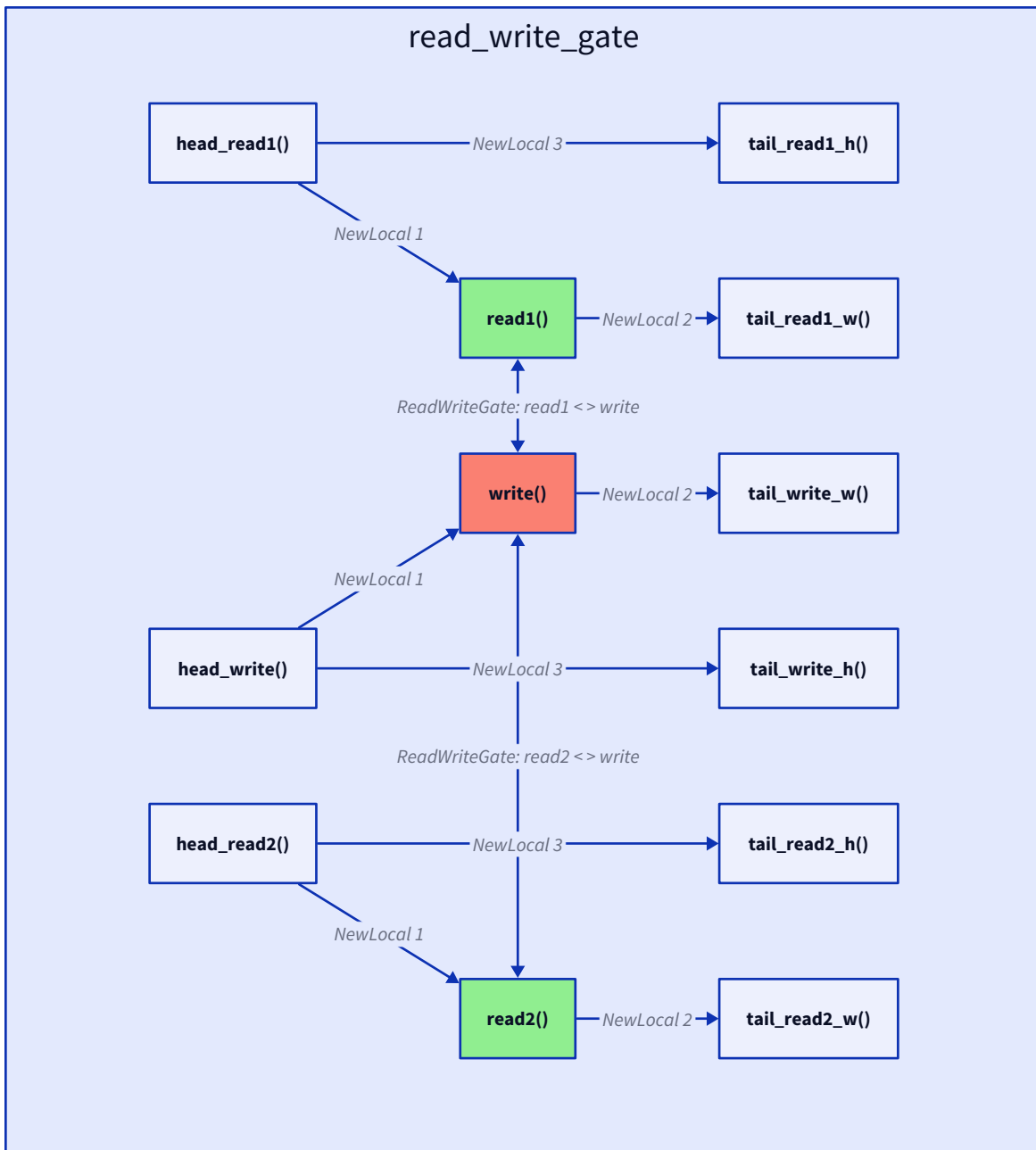
§ 3.2 `read_write_gate`

Similar to a `shared_mutex` , but for structured concurrency, a `read_write_gate` allows multiple shared operations to execute concurrently, while exclusive operations execute alone.

§ 3.2.1 Constraints

The `read_write_gate` follows the same split-tail model described for `serial_gate` , so the (*NewLocal*) constraints apply to each admitted operation.

The non-local constraint is (*SharedMutex*): read operations may overlap with other read operations, while write operations are ordered with respect to all other operations admitted through the same gate.



The constraints imposed by a read_write_gate.

§ 3.2.2 API

Following the `serial_gate` API, the primitive can expose separate entry senders for shared and exclusive admission:

```
struct read_write_gate {
    std::execution::enter_scope_sender acquire_read();
    std::execution::enter_scope_sender acquire_write();
};
```

The use of the gate is otherwise the same as for `serial_gate` :

```
read_write_gate g;
std::execution::counting_scope scope;

void read_usage() {
```

```

    head_read();

    auto s = std::execution::within(g.acquire_read(), read_sender())
        | std::execution::then([] { tail_read_w(); });

    std::execution::spawn(std::move(s), scope.get_token());

    tail_read_h();
}

void write_usage() {
    head_write();

    auto s = std::execution::within(g.acquire_write(), write_sender())
        | std::execution::then([] { tail_write_w(); });

    std::execution::spawn(std::move(s), scope.get_token());

    tail_write_h();
}

```

§ 3.2.3 Examples

§ 3.2.3.1 Concurrent cache reads with exclusive updates

A cache often allows many concurrent lookups, but requires updates to be exclusive.

```

read_write_gate cache_gate;
cache c;

auto lookup(key k) {
    auto read
        = std::execution::just(std::move(k))
        | std::execution::then([&](key k) {
            return c.lookup(k);
        });

    return std::execution::within(cache_gate.acquire_read(),
        std::move(read))
        | std::execution::then([](record r) {
            use_record(std::move(r));
        });
}

auto insert(record r) {
    auto write
        = std::execution::just(std::move(r))
        | std::execution::then([&](record r) {
            c.insert(std::move(r));
        });

    return std::execution::within(cache_gate.acquire_write(),
        std::move(write));
}

```

Multiple calls to lookup can execute concurrently. Calls to insert are serialized with respect to all other cache operations admitted by the gate.

§ 3.2.3.2 Updating shared configuration

The same pattern applies when many operations need a stable view of shared configuration, but configuration reloads must be exclusive.

```
read_write_gate config_gate;
configuration config;

auto serve(request req) {
    auto read_config
        = std::execution::just(std::move(req))
        | std::execution::then([&](request req) {
            return handle_with_config(std::move(req), config);
        });

    return std::execution::within(config_gate.acquire_read(),
        std::move(read_config));
}

auto reload(configuration next) {
    auto update_config
        = std::execution::just(std::move(next))
        | std::execution::then([&](configuration next) {
            config = std::move(next);
        });

    return std::execution::within(config_gate.acquire_write(), std::move(update_config));
}
```

Requests may read the current configuration concurrently. A reload is admitted only when no request is using the configuration through the gate, and no new request is admitted in read mode while the reload is active.

§ 3.2.4 Safety and progress

The safety and progress argument is the same as for `serial_gate`. The implementation maintains a different admission invariant: either any number of read operations are admitted and no write operation is admitted, or exactly one write operation is admitted and no read operation is admitted. As long as the submitted work preserves safety and progress, and the calling code maintains the lifetime of the gate and associated operations, a `read_write_gate` can maintain the read-write constraint without exposing a manual signaling protocol that lets users break progress independently of the submitted work.

The same eventual-admission requirement applies: an operation submitted to the gate should eventually be admitted when its admission mode is compatible with the operations ahead of it, assuming the operation is not cancelled or abandoned, the gate remains alive, previously admitted work eventually completes, and the scheduler provides progress. If a

fairness policy is not specified, the guarantee may need to be phrased in terms of the chosen admission policy; for example, a writer should not be indefinitely bypassed by later readers unless the specification explicitly permits that behavior.

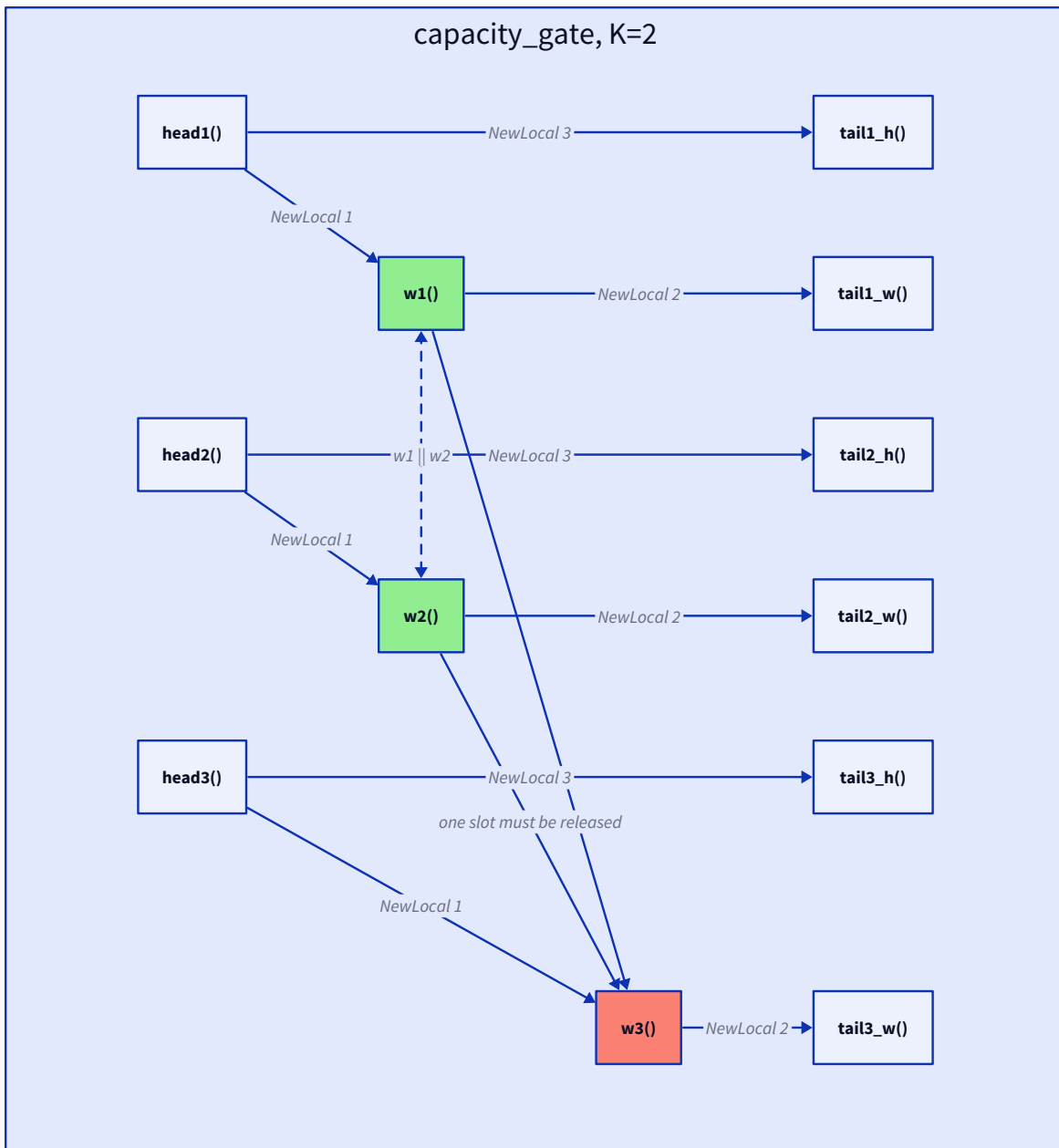
§ 3.3 **capacity_gate**

Similar to a `counting_semaphore`, but for structured concurrency, a `capacity_gate` allows at most K operations to execute their protected work concurrently.

§ 3.3.1 **Constraints**

The `capacity_gate` follows the same split-tail model described for `serial_gate`, so the (*NewLocal*) constraints apply to each admitted operation.

The non-local constraint is (*Semaphore*): among all operations admitted through the same gate, at most K protected work items may execute concurrently.



The constraints imposed by a capacity_gate with capacity 2.

§ 3.3.2 API

Following the serial_gate API, the primitive can expose one entry sender. The maximum concurrency is a property of the gate object.

```

struct capacity_gate {
    explicit capacity_gate(size_t max_concurrency);

    std::execution::enter_scope_sender acquire();
};
  
```

The use of the gate is otherwise the same as for serial_gate :

```

capacity_gate g{2};
std::execution::counting_scope scope;
  
```

```

void usage() {
    head();

    auto s = std::execution::within(g.acquire(), w_sender())
        | std::execution::then([] { tail_w(); });

    std::execution::spawn(std::move(s), scope.get_token());

    tail_h();
}

```

§ 3.3.3 Examples

§ 3.3.3.1 Limiting requests to a service

An application may need to limit the number of concurrent requests sent to an external service, while still allowing callers to construct sender expressions independently.

```

capacity_gate service_gate{8};
service client;

auto fetch(resource_id id) {
    auto request
        = std::execution::just(id)
        | std::execution::let_value([&](resource_id id) {
            return client.async_fetch(id);
        });

    return std::execution::within(service_gate.acquire(), std::move(re-
        quest));
}

```

At most eight calls to `client.async_fetch` are active through `service_gate` at any time. The caller of `fetch` can still compose the returned sender with other work before deciding how to start it.

§ 3.3.3.2 Bounding expensive CPU work

A `capacity_gate` can also bound work that is expensive even if the scheduler has more execution resources available.

```

capacity_gate compression_gate{4};

auto compress_file(path input, path output) {
    auto work
        = std::execution::just(std::move(input), std::move(output))
        | std::execution::then([](path input, path output) {
            compress(input, output);
        });

    return std::execution::within(compression_gate.acquire(),
        std::move(work));
}

```

Here the gate expresses a program-level concurrency limit, not a scheduling policy. The scheduler may have more than four worker threads, but no more than four compression operations admitted through the gate execute concurrently.

§ 3.3.4 Safety and progress

The safety and progress argument is the same as for `serial_gate`. The implementation maintains a different admission invariant: no more than K operations are admitted at once. As long as the submitted work preserves safety and progress, and the calling code maintains the lifetime of the gate and associated operations, a `capacity_gate` can maintain the bounded-concurrency constraint without exposing a manual signaling protocol that lets users break progress independently of the submitted work.

The same eventual-admission requirement applies: an operation submitted to the gate should eventually be admitted when capacity becomes available, assuming the operation is not cancelled or abandoned, the gate remains alive, previously admitted work eventually completes, and the scheduler provides progress.

The same considerations on fairness and starvation apply here. For ensuring progress under any circumstances, the users of capacity gate must provide stronger guarantees.

§ 3.4 Speculative execution

Similar to the classic `try_lock` facility, we can bring speculative execution to gates. Sometimes an operation is useful only if it can start immediately. If the non-local constraint would force the operation to wait, the program may prefer to skip that operation and continue with other work.

For sender-based gates, this can be expressed by adding `try_` versions of the corresponding acquire operations. A `try_acquire` operation attempts to enter the gate without enqueueing the operation as a waiter. If the operation can be admitted immediately, it completes successfully and produces the exit sender used by `within`. If the operation cannot be admitted immediately, it completes with `set_stopped()`.

For example, a `serial_gate` might provide:

```
struct serial_gate {
    std::execution::enter_scope_sender acquire();
    std::execution::enter_scope_sender try_acquire();
};
```

and a `read_write_gate` might provide:

```
struct read_write_gate {
    std::execution::enter_scope_sender acquire_read();
    std::execution::enter_scope_sender acquire_write();

    std::execution::enter_scope_sender try_acquire_read();
};
```

```
std::execution::enter_scope_sender try_acquire_write();  
};
```

This allows callers to express speculative work directly:

```
serial_gate cache_update_gate;  
  
auto opportunistic_refresh() {  
    return std::execution::within(  
        cache_update_gate.try_acquire(),  
        refresh_cache_snapshot());  
}
```

If the gate is free, `refresh_cache_snapshot()` executes under the gate. If the gate is occupied, the returned sender completes with `set_stopped()` and no refresh is performed.

The same idea applies to `capacity_gate`:

```
capacity_gate upload_gate{8};  
  
auto maybe_upload(chunk c) {  
    return std::execution::within(  
        upload_gate.try_acquire(),  
        upload_chunk(std::move(c)));  
}
```

Here an upload is started only if capacity is available immediately. If all capacity is already in use, the operation is stopped rather than queued.

The important property is that speculative execution does not weaken the gate's safety invariant. A successful `try_acquire` admits the operation exactly like the corresponding non-speculative acquire. An unsuccessful `try_acquire` does not admit the operation and therefore does not execute the protected work. Thus, the only additional behavior is the possibility of `set_stopped()` before the protected work starts.

Speculative acquisition is not a replacement for the eventual-admission guarantees discussed above. A normal acquire operation submits work to the gate and expects eventual admission or eventual resolution according to the gate's semantics. A `try_acquire` operation is explicitly different: it asks whether the work can be admitted now, and otherwise declines to wait.

§ 3.5 readiness_gate

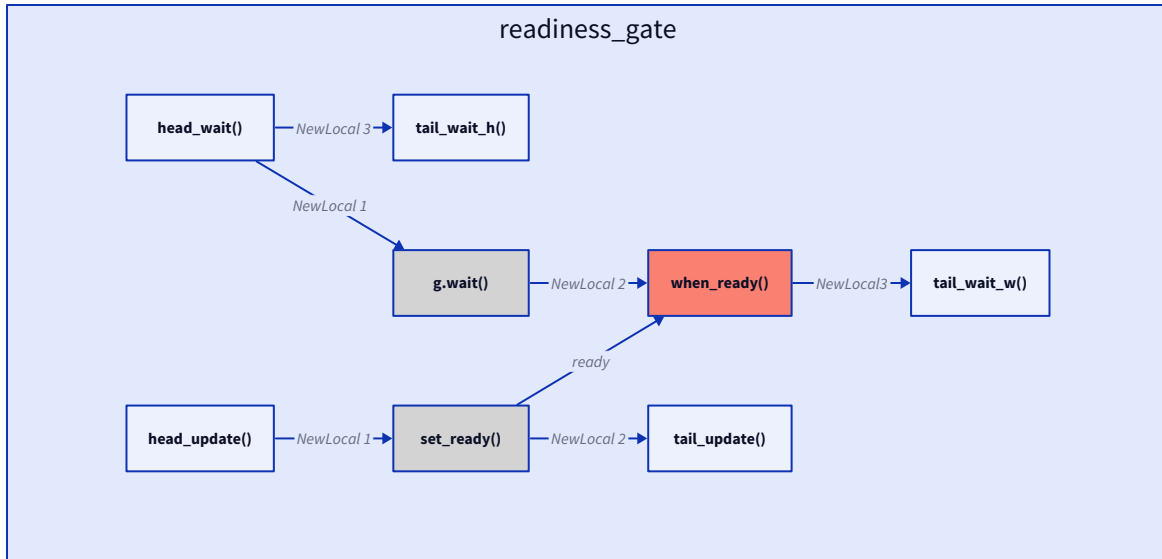
Similar to a `condition_variable`, but for structured concurrency, a `readiness_gate` delays work until some readiness condition has been established.

§ 3.5.1 Constraints

The `readiness_gate` follows the same split-tail model described for `serial_gate`, so the (*NewLocal*) constraints apply to the operations involved.

The non-local constraint is (*CondVar*): assuming the readiness condition is initially false, work that depends on the condition can start only after some operation has made the condition true.

Unlike a condition variable, the gate should not expose a protocol in which one operation waits and an unrelated operation must remember to notify it. The readiness transition is represented as sender work, and waiting operations are resumed when the gate observes that transition.



The constraints imposed by a `readiness_gate`.

§ 3.5.2 API

One possible API separates the operation that waits for readiness from the operation that establishes readiness:

```
struct readiness_gate {
    std::execution::enter_scope_sender wait();
    std::execution::sender auto set_ready();
    std::execution::sender auto close();
};
```

The `wait()` sender completes when the gate is ready and admits the dependent work. The `set_ready()` sender establishes readiness and resumes operations waiting on the gate. The `close()` sender completes waiting operations with `set_stopped()` if readiness has not been established.

```
readiness_gate g;
std::execution::counting_scope scope;

void wait_usage() {
    head_wait();

    auto s = std::execution::within(g.wait(), when_ready_sender())
        | std::execution::then([] { tail_wait_w(); });
}
```

```

    std::execution::spawn(std::move(s), scope.get_token());

    tail_wait_h();
}

void update_usage() {
    head_update();

    auto s = g.set_ready()
        | std::execution::then([] { tail_update_w(); });

    std::execution::spawn(std::move(s), scope.get_token());

    tail_update_h();
}

```

§ 3.5.3 Alternative API: predicate-owned readiness

The API above treats readiness as an explicit transition: some operation decides that the condition is true and calls `set_ready()`. This is simpler than a `condition_variable`, but it does not model the traditional condition-variable pattern in which the waiting operation re-checks a predicate protected by the same synchronization mechanism. In that pattern, notification is only a hint; the predicate is the source of truth.

An alternative design is to make the predicate part of the gate:

```

template <class Predicate>
struct readiness_gate {
    explicit readiness_gate(Predicate pred);

    std::execution::enter_scope_sender wait();
    std::execution::sender auto update();
    std::execution::sender auto close();
};

```

In this design, `wait()` admits dependent work only when the predicate evaluates to `true`. The `update()` operation is used after code has modified the state observed by the predicate; it causes the gate to re-check the predicate and admit waiters if the condition is now satisfied. If the gate is closed before the predicate becomes true, waiters complete with `set_stopped()`.

For example:

```

configuration config;

readiness_gate config_loaded{[&] {
    return config.has_value();
}};

auto load_configuration(path p) {
    return async_read_config(std::move(p))
        | std::execution::then([&](configuration loaded) {
            config = std::move(loaded);
        });
}

```

```

    })
    | std::execution::let_value([&] {
        return config_loaded.update();
    });
}

auto handle_request(request req) {
    auto use_config
    = std::execution::just(std::move(req))
    | std::execution::then([&](request req) {
        return handle_with_config(std::move(req), config);
    });

    return std::execution::within(config_loaded.wait(), std::move(use_config));
}

```

This alternative is closer to `condition_variable`: the predicate, not the update operation itself, determines whether dependent work may proceed. It avoids admitting work after an incorrect `set_ready()` call, and it can naturally handle cases in which multiple updates are needed before readiness is established.

The cost is that the gate now has to own or reference the predicate and define where and how the predicate is evaluated. If the predicate reads shared state, the design must also specify how that state is protected from concurrent access. This may require combining `readiness_gate` with another gate, or making the readiness gate itself responsible for the state being checked. That makes the abstraction heavier than the explicit-transition API.

The explicit `set_ready()` design is appropriate when readiness is a one-shot fact established by a specific operation, such as successful initialization. The predicate-owned design is appropriate when readiness is derived from shared state and the gate must prevent users from separating the readiness signal from the condition that justifies it.

§ 3.5.4 Examples

§ 3.5.4.1 Waiting for initialization

One common use is to delay work until asynchronous initialization has completed.

```

readiness_gate initialized;
service svc;

auto start_service(configuration cfg) {
    auto start
    = std::execution::just(std::move(cfg))
    | std::execution::let_value([&](configuration cfg) {
        return svc.async_start(std::move(cfg));
    })
    | std::execution::let_value([&] {
        return initialized.set_ready();
    });
}

```

```

    return start;
}

auto query_service(query q) {
    auto query
        = std::execution::just(std::move(q))
        | std::execution::let_value([&](query q) {
            return svc.async_query(std::move(q));
        });

    return std::execution::within(initialized.wait(), std::move(query));
}

```

Calls to `query_service` may be created before the service has started, but their queries are not admitted until `start_service` establishes readiness.

§ 3.5.4.2 Waiting for configuration

A component may also need to delay requests until the first configuration value has been loaded.

```

readiness_gate config_loaded;
configuration config;

auto load_configuration(path p) {
    auto load
        = async_read_config(std::move(p))
        | std::execution::then([&](configuration loaded) {
            config = std::move(loaded);
        })
        | std::execution::let_value([&] {
            return config_loaded.set_ready();
        });

    return load;
}

auto handle_request(request req) {
    auto use_config
        = std::execution::just(std::move(req))
        | std::execution::then([&](request req) {
            return handle_with_config(std::move(req), config);
        });

    return std::execution::within(config_loaded.wait(), std::move(use_config));
}

```

The gate expresses that request handling depends on the initial configuration becoming available. Once readiness is established, subsequent requests can proceed without a blocking wait.

§ 3.5.5 Safety and progress

The safety and progress argument follows the same ideas as for `serial_gate`. The implementation maintains a different admission invariant: dependent work is not admitted until readiness has been established. As long as the work that establishes readiness and the work that depends on readiness preserve safety and progress, and the calling code maintains the lifetime of the gate and associated operations, a `readiness_gate` can maintain the readiness constraint without exposing a condition-variable-style manual notification protocol.

The eventual-admission requirement becomes an eventual-resolution requirement for `readiness_gate`: an operation submitted to the gate should eventually either be admitted after readiness is established, or complete with `set_stopped()` if the gate is closed before readiness is established. This assumes that the operation is not otherwise cancelled or abandoned, the gate remains alive until it is made ready or closed, and the scheduler provides progress. Before readiness is established or the gate is closed, waiting is not a progress failure; it is the constraint expressed by the gate.

The presence of `close()` also has implications for the destruction of `readiness_gate` objects; see [Destruction](#) for more details on the requirements for destruction.

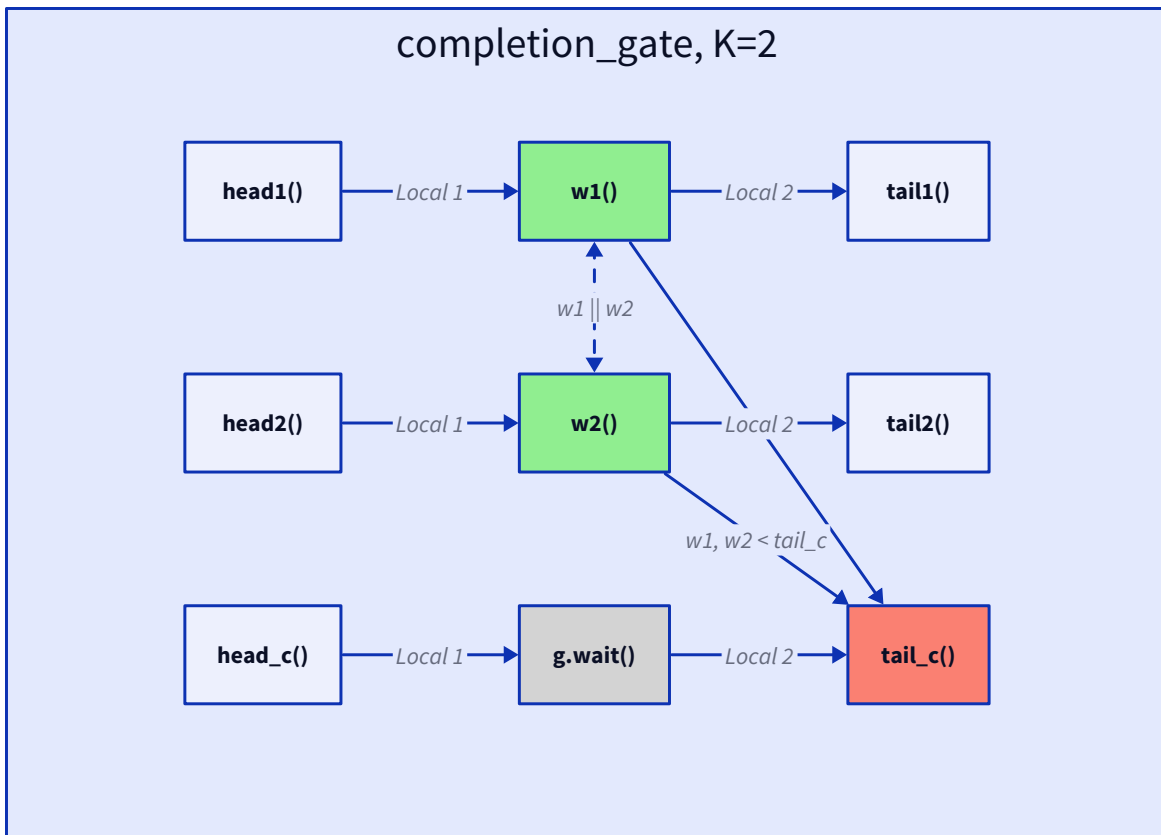
§ 3.6 `completion_gate`

Similar to a `latch`, but for structured concurrency, a `completion_gate` delays dependent work until a fixed number of submitted operations have completed.

§ 3.6.1 Constraints

The `completion_gate` follows the same split-tail model described for `serial_gate`, so the (*NewLocal*) constraints apply to the operations involved.

The non-local constraint is (*Latch*): dependent work admitted through the gate can start only after all operations that contribute to opening the gate have completed. As with `latch`, the contributing operations are not ordered with respect to each other by the gate.



The constraints imposed by a completion_gate.

§ 3.6.2 API

One possible API separates arrivals from waiting for completion:

```

struct completion_gate {
    explicit completion_gate(size_t expected);

    std::execution::sender auto arrive();
    std::execution::enter_scope_sender wait();
    std::execution::sender auto close();
};
  
```

The `arrive()` sender records one completion. The `wait()` sender completes when the expected number of arrivals has been recorded and admits the dependent work. The `close()` sender completes waiting operations with `set_stopped()` if the expected number of arrivals has not been reached. In typical use, `arrive()` is structurally attached to the completion of the contributing sender, as in the examples below.

```

completion_gate g{2};
std::execution::counting_scope scope;

void worker1() {
    head1();

    auto s
        = w1_sender()
        | std::execution::let_value([&] {
            return g.arrive();
        });
  
```

```

    })
    | std::execution::then([] { tail1_w(); });

std::execution::spawn(std::move(s), scope.get_token());

tail1_h();
}

void coordinator() {
    head_c();

    auto s = std::execution::within(g.wait(), after_completion_sender())
        | std::execution::then([] { tail_c_w(); });

    std::execution::spawn(std::move(s), scope.get_token());

    tail_c_h();
}

```

§ 3.6.3 Examples

§ 3.6.3.1 Waiting for independent startup tasks

Several independent startup tasks may need to complete before the rest of a service can begin accepting work.

```

completion_gate startup_done{3};

auto load_index() {
    return async_load_index()
        | std::execution::let_value([&] {
            return startup_done.arrive();
        });
}

auto connect_database() {
    return async_connect_database()
        | std::execution::let_value([&] {
            return startup_done.arrive();
        });
}

auto warm_cache() {
    return async_warm_cache()
        | std::execution::let_value([&] {
            return startup_done.arrive();
        });
}

auto accept_requests() {
    return std::execution::within(startup_done.wait(),
        start_accepting_requests());
}

```

The three startup tasks can run concurrently. `accept_requests` is admitted only after all three arrivals have been recorded.

§ 3.6.3.2 Joining a fixed batch

A `completion_gate` can also express that a continuation depends on a fixed batch of operations, without imposing order between the operations in the batch.

```
completion_gate batch_done{files.size()};

auto process_file(path p) {
    return async_process_file(std::move(p))
        | std::execution::let_value([&] {
            return batch_done.arrive();
        });
}

auto write_summary() {
    return std::execution::within(batch_done.wait(), async_write_summary());
}
```

Each file-processing operation contributes one arrival. The summary is not written until all file-processing operations in the batch have completed.

§ 3.6.4 Safety and progress

The safety and progress argument follows the same ideas as for `serial_gate`. The implementation maintains a different admission invariant: dependent work is not admitted until the required number of arrivals has been recorded. As long as the arriving work and the dependent work preserve safety and progress, and the calling code maintains the lifetime of the gate and associated operations, a `completion_gate` can maintain the completion constraint without exposing a manual wait/notify protocol.

The eventual-admission requirement becomes an eventual-resolution requirement for `completion_gate`: an operation submitted to the gate should eventually either be admitted after the expected number of arrivals has been recorded, or complete with `set_stopped()` if the gate is closed before that happens. This assumes that the operation is not otherwise cancelled or abandoned, the gate remains alive until it is opened or closed, and the scheduler provides progress. Before the expected number of arrivals has been recorded or the gate is closed, waiting is not a progress failure; it is the constraint expressed by the gate.

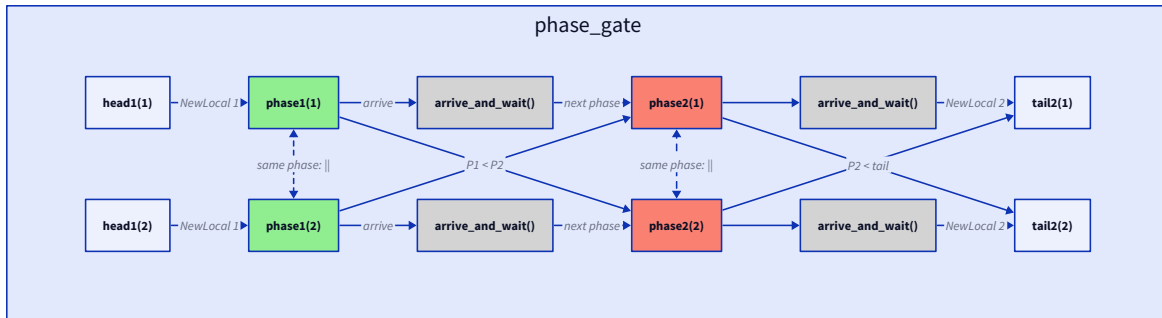
The presence of `close()` also has implications for the destruction of `completion_gate` objects; see [Destruction](#) for more details on the requirements for destruction.

§ 3.7 phase_gate

Similar to a `barrier`, but for structured concurrency, a `phase_gate` coordinates a fixed set of participants through repeated phase boundaries.

§ 3.7.1 Constraints

The `phase_gate` follows the same split-tail ideas as the previous gates, but the important relation is between phases. The non-local constraint is (*Barrier*): all work in one phase must complete before any work in the next phase can execute. Work within the same phase is not ordered by the gate and may execute concurrently.



The constraints imposed by a `phase_gate`.

§ 3.7.2 API

The central operation for a phase gate is an asynchronous `arrive_and_wait()` operation. It records the participant's arrival at the current phase and completes only when all participants for that phase have arrived.

```
struct phase_gate {
    explicit phase_gate(size_t expected);

    std::execution::enter_scope_sender arrive_and_wait();
    std::execution::sender auto close();
};
```

The `arrive_and_wait()` operation admits the continuation after the phase boundary. The `close()` sender completes waiting operations with `set_stopped()` if the phase cannot complete.

```
phase_gate g{2};
std::execution::counting_scope scope;

void participant() {
    auto s
        = phase1_sender()
        | std::execution::let_value([&] {
            return std::execution::within(g.arrive_and_wait(), phase2_sender());
        });

    std::execution::spawn(std::move(s), scope.get_token());
}
```

§ 3.7.3 Examples

§ 3.7.3.1 Iterative computation

An iterative algorithm may have multiple participants that compute a step independently, then exchange or observe the results only after all participants have completed the step.

```
phase_gate iteration_done{workers.size()};

auto worker(worker_state& state) {
    return compute_step(state)
        | std::execution::let_value([&] {
            return std::execution::within(
                iteration_done.arrive_and_wait(),
                exchange_boundaries(state));
        })
        | std::execution::let_value([&] {
            return std::execution::within(
                iteration_done.arrive_and_wait(),
                compute_next_step(state));
        });
}
```

The first `arrive_and_wait()` ensures that no participant exchanges boundaries before all participants have completed `compute_step`. The second one ensures that no participant starts the next step before all participants have completed the boundary exchange.

§ 3.7.3.2 Coordinated pipeline stages

A group of participants may also need to move through pipeline stages together, while allowing concurrency inside each stage.

```
phase_gate stage_done{participants};

auto participant(input_chunk input) {
    return parse_to_local_storage(std::move(input))
        | std::execution::let_value([&] {
            return std::execution::within(
                stage_done.arrive_and_wait(),
                validate_local_storage());
        })
        | std::execution::let_value([&] {
            return std::execution::within(
                stage_done.arrive_and_wait(),
                publish_local_storage());
        });
}
```

All participants complete parsing before any participant validates, and all participants complete validation before any participant publishes.

§ 3.7.4 Safety and progress

The safety and progress argument follows the same ideas as for `serial_gate`. The implementation maintains a different admission invariant: continuations after a phase boundary are not admitted until all expected participants have arrived at that boundary. As long as

each participant's phase work preserves safety and progress, and the calling code maintains the lifetime of the gate and associated operations, a `phase_gate` can maintain the phase-ordering constraint without exposing a blocking barrier wait.

The eventual-admission requirement becomes an eventual-resolution requirement for `phase_gate`: an operation submitted to the gate should eventually either be admitted after all expected participants have arrived at the current phase, or complete with `set_stopped()` if the gate is closed before that happens. This assumes that the operation is not otherwise cancelled or abandoned, the gate remains alive until the phase is completed or closed, and the scheduler provides progress. Before all expected participants arrive or the gate is closed, waiting is not a progress failure; it is the constraint expressed by the gate.

The presence of `close()` also has implications for the destruction of `phase_gate` objects; see [Destruction](#) for more details on the requirements for destruction.

§ 3.8 Destruction

All gate objects have lifetime requirements. A gate owns the state needed to remember waiting operations, admission order, and the operations currently admitted through the gate. Destroying that state while operations are still associated with it would leave those operations without a well-defined synchronization object.

For this reason, destroying a gate while it has outstanding associated work is a program error. The implementation should call `std::terminate()` in this case. Outstanding work includes operations that have been submitted to the gate but not yet admitted, operations currently admitted whose exit sender has not completed, and operations waiting for the gate to resolve a dependency.

This rule is intentionally strict. A gate destructor should not try to block until outstanding work completes, because blocking may consume an execution resource needed by that work to make progress. It also should not silently abandon operations, because that would make progress depend on object lifetime in a way that is difficult to reason about. Programs that need to shut down a gate must first arrange for all associated operations to complete, be cancelled, or otherwise be resolved by the gate's explicit API, and only then destroy the gate.

For gates with an explicit closing operation, such as `readiness_gate::close()`, closing the gate is separate from destroying it. Closing resolves operations according to the gate's semantics; destruction still requires that no outstanding associated work remains.

§ 3.9 Beyond classic-inspired gates

The gates above are derived from existing synchronization primitives. Once admission is expressed as sender composition, other useful non-local constraints become possible.

- A `keyed_serial_gate` serializes operations that have the same key, while allowing operations with different keys to proceed concurrently.
- A `budget_gate` admits operations while a resource budget is available; unlike `capacity_gate`, different operations may consume different amounts.
- A `priority_gate` admits pending operations according to priority.
- A `deadline_gate` admits, orders, or stops pending operations according to deadlines.
- A `rate_gate` admits operations according to a time-based rate rather than a concurrency bound.
- A `throttle_gate` spaces out admitted operations, for example by ensuring a minimum interval between starts.
- A `backpressure_gate` admits upstream work only when downstream capacity is available.
- A `coalescing_gate` combines compatible pending operations before admission.
- A `latest_only_gate` retains only the most recent pending operation and stops or discards older pending operations that have been superseded.

§ 4 Relation to other facilities

§ 4.1 P3955R0: asynchronous scopes

The APIs above are intentionally phrased in terms of the scope vocabulary from [\[P3955R0\]](#). In that model, entering an asynchronous scope is itself a sender operation, and leaving the scope is represented by an exit sender that is structurally connected to the completion of the work executed inside the scope.

That is exactly the shape needed by gates. A gate's `acquire()` operation is an enter-scope sender: it waits until the gate can admit the operation, and then produces the exit sender that releases the gate. The expression

```
std::execution::within(g.acquire(), snd)
```

therefore means: enter the gate, run `snd`, and release the gate after `snd` completes. This is what prevents the gate from degenerating into a manual acquire/release protocol. The release operation is not a separate user action; it is part of the structure of the sender expression.

The same observation applies directly to gates that admit work into a protected region, such as `read_write_gate::acquire_read()`, `read_write_gate::acquire_write()`, and `capacity_gate::acquire()`. For dependency and rendezvous gates, such as `completion_gate::wait()` and `phase_gate::arrive_and_wait()`, the enter-scope shape is still useful, but the exit sender may be trivial: the important operation is delaying admission of the dependent work until the gate's condition is satisfied.

Thus, this paper can be seen as exploring concrete asynchronous scopes whose purpose is not object lifetime, but non-local concurrency control.

§ 4.2 counting_scope

The `counting_scope` facility from [P3149R11] solves a different but related problem. A `counting_scope` tracks the lifetime of asynchronous work that has been associated with the scope. It gives a program a way to spawn non-sequential work and later close, stop, and join that work.

This is already a primitive for non-local concurrency. Work associated with a `counting_scope` is not necessarily lexically nested in the caller that started it, but the scope still imposes non-local lifetime constraints:

- the scope remembers outstanding work;
- closing the scope prevents new work from being associated;
- joining the scope waits for associated work to finish;
- destroying the scope with outstanding work is a contract violation.

The gates proposed in this paper are complementary. A gate controls *when* work may execute; a `counting_scope` controls *how long* spawned work remains associated with an enclosing lifetime. In the examples above, these roles are often combined:

```
auto protected_work =  
    std::execution::within(g.acquire(), work());  
  
std::execution::spawn(std::move(protected_work), scope.get_token());
```

Here the gate imposes the serialization, read-write, capacity, readiness, completion, or phase constraint. The `counting_scope` tracks the lifetime of the spawned operation. Neither facility subsumes the other.

There is also a design connection through `scope_token`. One possible API for gates is to expose a token that models the same token concept used by `counting_scope`; the token's `wrap` operation would apply the gate constraint to the associated sender. This paper does not rely on that spelling, but the relationship suggests that gates and `counting_scope` may share customization and specification machinery.

§ 4.3 Other related facilities

The following facilities are also adjacent to this design space:

- **Schedulers** decide where and when execution resources are used. Gates do not replace schedulers; they impose additional admission constraints on work that may later run on some scheduler. Conversely, a scheduler can implement some admission policies for work that it owns, but it cannot generally implement sender-scoped gate semantics: it may observe that work has been started on an execution resource, but it

does not generally know when the sender operation protected by the gate has completed and the next operation can be admitted.

- **Sender algorithms** such as `when_all` , `let_value` , `starts_on` , and `continues_on` express local sender composition. Gates add shared state that allows different sender expressions to participate in the same non-local constraint.
- **Classic synchronization primitives** remain useful for synchronous code. Gates are not wrappers around blocking primitives; they are sender-native forms of the same kinds of ordering and admission constraints.

§ 5 The ask for SG1

This paper is exploratory. The design of various facilities still needs work. It does not yet ask SG1 to standardize any particular gate. Instead, it asks whether this is a useful direction for sender-based concurrency, and where further work should be focused.

In particular, we would like SG1 feedback on the following questions:

- Does the *gate* model make sense as a way to express non-local concurrency constraints in sender-based programs?
- Are the proposed goals the right ones: compositional safety, compositional progress, non-blocking waiting, and explicit constraints?
- Is the connection to [P3955R0] the right foundation for these primitives, or should gates be expressed through a different sender vocabulary?
- Is the separation between gates and `counting_scope` from [P3149R11] clear and useful?
- Which of the classic-inspired gates are most worth pursuing first: `serial_gate` , `read_write_gate` , `capacity_gate` , `readiness_gate` , `completion_gate` , or `phase_gate`?

The main question is whether SG1 agrees that C++ needs sender-native primitives for these non-local constraints. If so, the next question is where the author should spend effort: formalizing one or two concrete gates, refining the common gate model, exploring integration with scope tokens, or developing more examples and implementation experience.

§ 6 References

[P3149R11] Ian Petersen, Jessica Wong; Dietmar Kühl; Ján Ondrušek; Kirk Shoop; Lee Howes; Lucian Radu Teodorescu; Ruslan Arutyunyan; 2025-06-19. `async_scope` — Creating scopes for non-sequential concurrency.

<https://wg21.link/p3149r11>

[P3955R0] Robert Leahy. 2026-01-16. It's Scopes All the Way Down.

<https://wg21.link/p3955r0>

[P4214R0] Lucian Radu Teodorescu. 2026-05-12. Composable Correctness and Progress Guarantees.

<https://wg21.link/p4214r0>

[Teodorescu26] Lucian Radu Teodorescu. 2026-04. Elements of Concurrency. *Overload* 34, 192 (April 2026), 4–9.

<https://accu.org/journals/overload/34/192/overload192.pdf#page=6>